

Table of Content

1. Introduction	1
2. Dataset	2
3. Data Organization	2
4. Data Preprocessing	3
5. Methods	4
5.1 Exact Jaccard Similarity	4
5.2 MinHash Approximation	5
5.3 Locality Sensitive Hashing (LSH)	5
5.4 TF-IDF and Cosine Similarity	6
6. Experiments and Results	6
6.1 Exact Jaccard Similarity	6
6.2 MinHash Approximation	7
6.3 Locality Sensitive Hashing	7
6.4 TF-IDF and Cosine Similarity	7
7. Scalability Analysis	8
8. Discussion	9
9. Conclusion	9

1. Introduction

Document similarity represents one of the fundamental problems in information retrieval and natural language processing. Identification of similar documents is crucial in applications like duplicate detection, recommendation systems, plagiarism detection, clustering and search engines. Even though exact similarity measures can provide very precise results, they are prohibitively computationally expensive in large datasets. Scalable approximations are thus utilized in modern information retrieval systems.

This project is intended to explore and compare several methods for calculating document similarity using a real-world corpus of news articles. The focus will be put on both the precision of finding similar documents and the efficiency of the approaches used, paying special attention to the difference between exact similarity calculation and scalable approximation algorithms.

Four complementary methods are implemented. First, exact Jaccard Similarity is used as the reference method, comparing sets of words from news articles. Second, MinHash signatures are calculated to efficiently approximate Jaccard similarity without full comparison of token sets. Third, Locality Sensitive Hashing (LSH) is applied to retrieve candidate documents and avoid exhaustive pairwise comparisons. Fourth, TF-IDF vectorization along with Cosine Similarity is utilized to find document similarity based on term importance rather than lexical overlap.

All the algorithms are implemented in Python using Jupyter Notebook. They are evaluated on The New York Times articles published in 2020. The focus is on data preprocessing, the quality of found neighbors, and the scalability of the implemented approaches. The implementation meets the key requirement of the project: the proposed techniques must be applicable to large document corpora and computationally feasible.

2. Dataset

Experiments use the The New York Times Articles 2020 dataset, consisting of articles published by The New York Times in 2020. Each record in the dataset includes various textual components, such as titles, abstracts, publication dates, and other metadata. For the purpose of the current research, only the abstracts were used. Abstracts represent a concise summary of an article containing enough semantic information to assess document similarity while reducing the amount of computations needed to process full texts.

The dataset consists of 16,787 articles. Each abstract is treated as a separate document in the dataset, thereby providing a sufficiently large set to test both exact and scalable approximation-based similarity measures. The dataset is imported to Python via pandas library and saved as a DataFrame. Only necessary parts of metadata are used in preprocessing and ignored in calculations as they do not have any effect on the document similarity.

	newdesk	section	subsection	material	headline	abstract	keywords	word_count	pub_date	is_popular	uniqueID
0	OpEd	Opinion	NaN	Op-Ed	Anyone Else Want to See Trump 'Shut Up'?	Our president as a terrible toddler.	['Presidential Election of 2020', 'Siden, Jose...	925	2020-10-01 00:05:51+0000	1	nyt://article/467c2ae-2d89-5d36-v6ca-b23d0433...
1	OpEd	Opinion	NaN	Op-Ed	Trump Calls on Extremists to 'Stand By'	Instead of condemning violent groups, the pres...	['Presidential Election of 2020', 'United Stat...	902	2020-10-01 00:43:28+0000	1	nyt://article/9a7ef9e0-1334-5d62-a7f1-288c4887...
2	OpEd	Opinion	NaN	Op-Ed	Can Mike Espy Make History, Again?	If the Democratic Party claims to value Black ...	['Black People', 'Blacks', 'Presidential Elect...	1412	2020-10-01 00:45:17+0000	1	nyt://article/4b2b763-0088-5e10-b204-19e404f7...
3	Games	Crosswords & Games	NaN	News	In Which Kikishi Wear Mawashi	Adam Fromm is on the line.	['Crossword Puzzles']	849	2020-10-01 02:00:05+0000	1	nyt://article/d996209f-edb8-51f1-8c44-1dd9ed5...
4	Sports	Sports	Pro Football	News	N.F.L. Week 4 Predictions: Our Picks Against L...	Tom Brady and the Buccaneers are building mome...	['Football', 'New England Patriots', 'Kansas C...	2690	2020-10-01 04:01:16+0000	0	nyt://article/af6295b-3c22-5a3f-9539-9f77b768...
...	...	...	...	...	...	...	...	...	...	...	...
3990	Editorial	Opinion	NaN	Op-Ed	What It Takes to Heal From Covid-19	Survivors can get better, but they need help.	['Chronic Condition (Health)', 'Coronavirus (2...	1002	2020-12-31 15:27:47+0000	1	nyt://article/relad8b75-a8b3-5a3c-68db-b9c1195f...
3991	Sports	Sports	Baseball	News	Padres Jolt M.L.B. With Bold Moves to Set Up W...	While many teams continued to assess the finan...	['San Diego Padres', 'Major League Baseball', ...	1100	2020-12-31 15:47:44+0000	0	nyt://article/1f11417d-3c57-51b9-b75d-99678a8...
3992	Business	Business Day	NaN	News	Their Finances Ravaged, Customers Fear Banks W...	Banks have the power to decide whether to let ...	['Banking and Financial Institutions', 'Corona...	1429	2020-12-31 16:21:40+0000	1	nyt://article/c49edab-bd0e-5d81-b496-9e6ed852...
3993	Dining	Food	Wine, Beer & Cocktails	News	Should Wine Be Among Your Health Resolutions?	The new category of 'clean wines' is an effort...	['Wines', 'Grapes', 'Diet and Nutrition', 'Dia...	1307	2020-12-31 17:28:11+0000	1	nyt://article/efcaf652-fdad-5b4e-9f17-4d8aaf5...
3994	Business	Technology	NaN	News	Microsoft Says Russian Hackers Viewed Some of ...	The hackers gained more access than the compan...	['Microsoft Corp', 'US Federal Government Data...	340	2020-12-31 18:02:02+0000	1	nyt://article/12048b2b-62e3-5bed-8c77-483a4299...

Figure 1. Preview of The New York Times Articles 2020 dataset. The figure shows the structure of the dataset and metadata included in each article, like headline, abstract, publication date and other descriptive data.

Before applying similarity algorithms the abstracts undergo cleaning and normalizing to produce the representations necessary for calculation of similarity measures.

3. Data Organization

The project was written in Python using a Jupyter Notebook. Libraries used include pandas for data manipulation, NumPy for numerical computations, spaCy for natural language preprocessing, scikit-learn for TF-IDF vectorization and cosine similarity, and datasketch for MinHash and Locality Sensitive Hashing (LSH).

The dataset was imported into a pandas DataFrame with each row representing a single article. Since the task of the current research is to identify similar textual documents, only the abstracts are retained, and other metadata, such as the publication date and URL, are dropped because they do not affect the similarity computations.

To support various similarity algorithms, multiple text representations were produced during preprocessing. Each article is saved as a list of normalized words and as a set of unique tokens. The first representation is used in the MinHash algorithm, while the second one is for Jaccard Similarity and TF-IDF vectorization. Thus, the same set of documents can be used for both set-based and vector space similarity algorithms without additional preprocessing steps.

4. Data Preprocessing

Textual data preprocessing is an important step in document similarity problems because raw texts include lots of grammatical variance, punctuation, stop words, and other elements that do not provide any semantic information. Hence, each abstract was preprocessed before the similarity algorithms were applied.

```
def preprocessing(document):  
    try:  
        word_set = set()  
        document = nlp(document)  
        for word in document:  
            lemma = word.lemma_.strip().lower()  
  
            if (  
                lemma  
                and lemma not in stopwords  
                and word.is_alpha  
                and not word.is_punct  
                and not word.is_space  
            ):  
                word_set.add(lemma)  
        return word_set  
    except:  
        return set()
```

Figure 2. Textual data preprocessing pipeline implemented with spaCy. The pipeline performs text transformation to the lower case, tokenization, removing punctuation, stop words, numbers and only leaves alphabetic tokens; the latter are lemmatized.

The preprocessing was done with the spaCy library and the English model en_core_web_sm. Each abstract goes through the following transformations:

- conversion to the lower case;
- tokenization via spaCy;
- removal of punctuation;
- removal of stop words;
- removal of numerical tokens;
- removal of non-alphabetic tokens;
- lemmatization of the rest tokens.

Lemmatization transforms grammatical forms of a word to its base form (like running, runs, ran → run), hence reducing vocabulary size and improving similarity measures.

After preprocessing, two different representations of each document were produced:

- Set representation of the unique lemmas for usage in Jaccard Similarity and MinHash, because these measures are based on sets rather than term frequencies;

- Normalized text where the lemmas are joined together as a string to be used in TF-IDF vectorization to keep term frequencies.

```
abstract1 = abstract.abstract.iloc[0]
abstract1
```

```
'Congress could do much more to protect Americans who have served their country from predatory for-profit colleges.'
```

```
abstract.processed[:10]
```

```
0    {much, congress, profit, serve, country, preda...
1    {christina, iverson, ring, new, jeff, chen, year}
2    {one, pass, meteor, major, long, spot, shower,...
3    {eclipse, miss, meteor, rocket, space, event, ...
4    {month, low, full, space, end, orbit, beyond, ...
5    {much, deploy, support, embassy, make, attack,...
6    {son, want, moola, end, brother, law, make, su...
7    {body, answer, would, woman, find, heart, left...
8    {juicy, eat, herb, leaf, tangy, joy, fill, sea...
9    {like, shariatmadari, word, quirk, snobbery, e...
Name: processed, dtype: object
```

Figure 3. Example of text preprocessing before the similarity computation. The preprocessing stage drops unnecessary tokens and produces two different normalized representations of the text.

The preprocessing greatly reduced the noise in the texts and preserved the semantic information allowing to apply similarity algorithms to the standardized text representations.

5. Methods

5.1 Exact Jaccard Similarity

The first similarity measure that was implemented is the Jaccard Similarity coefficient that represents an exact way to measure similarity for documents represented as sets of words. Contrary to frequency-based measures, Jaccard Similarity takes into account only the existence or nonexistence of unique terms, hence it is more suitable for lexical overlap.

Each document, after preprocessing, is converted to a set of its unique lemmatized tokens. Let A and B are token sets of two documents. The Jaccard Similarity is defined as $J(A,B) = |A \cap B| / |A \cup B|$, where $|A \cap B|$ is the number of common tokens and $|A \cup B|$ is the number of tokens occurring in any of the documents.

The value of J is in the range [0, 1], where 0 indicates no common tokens and 1 indicates equal token sets. In the implementation, each document is converted to the Python set of its tokens, and pairwise Jaccards are calculated using set operations. The retrieval procedure calculates similarities of a specified query document with all other documents and returns the results sorted in descending order of similarity.

An example from the evaluation shows that comparing article 10 with article 10532 yields $J = 0.142857$, indicating that about 14% of the vocabulary is common after preprocessing.

The Jaccard coefficient is easy to understand, but its computational complexity is quadratic to the number of documents, as all pair-wise comparisons are needed. Hence exhaustive Jaccard similarity is

inefficient for large document collections, however, it serves as an important reference point for other approximate algorithms.

5.2 MinHash Approximation

To overcome the computational problem of computing Jaccard similarity, MinHash was used to approximate the probabilistic Jaccard similarity using document signatures rather than full sets of their tokens.

In the current implementation, MinHash signatures were produced using the datasketch library. The tokens of each document were passed sequentially to the MinHash object, using 128 permutations (`num_perm = 128`) to balance the accuracy and efficiency.

The similarity is estimated directly from the MinHash signatures since the probability of the signatures equality corresponds to the Jaccard Similarity of the sets of tokens of documents. The retrieval procedure finds the most similar documents based on these estimates. Thus, for example, using article 10 as the query document, article 2952 is found to be the closest document, with an estimated similarity of 0.203125.

```
Checking minhash similarity for random articles

: minhash_check = minhash_sim_for_main(minhashes_list, 10)

minhash_check

[(2952, 0.203125),
 (2526, 0.1328125),
 (10532, 0.1328125),
 (12416, 0.1171875),
 (2988, 0.109375)]

: abstract.processed.iloc[[10, 2952, 2526]].values

: array([[{'eat', 'step', 'intermittent', 'less', 'try', 'fasting', 'drink', 'oyster', 'alcohol'},
         {'balanced', 'designer', 'intermittent', 'yam', 'proponent', 'fasting', 'soup', 'rely', 'bright', 'drink', 'stay', 'herbal', 'simple'},
         {'try', 'easy'}]], dtype=object)

Results are correct.
```

Figure 5. Top-ranked similar documents identified using MinHash signatures. The query document retrieves articles with the highest estimated Jaccard similarity based on compact MinHash representations.

Qualitative validation through manual checking confirmed a high degree of overlap among the retrieved documents, thereby verifying the meaningfulness of the MinHash results. Compared to the exhaustive Jaccard similarity, MinHash decreases the memory and comparison costs due to compact signatures and increases scalability.

5.3 Locality Sensitive Hashing (LSH)

Even though MinHash decreases the computation cost of similarity, the comparison of all pairs of signatures is still extensive for large datasets. Locality Sensitive Hashing (LSH) is used as an indexing technique for fast approximate nearest neighbor search, which further improves efficiency.

The idea of LSH is to group similar MinHash signatures into the same buckets increasing the probability of similar documents to collide in the same bucket. Therefore, the similarity computations are performed only for a subset of candidate documents extracted from the LSH index.

In the current research, MinHashLSH implementation of the datasketch library was used. All MinHash signatures that were generated before are inserted into the index with their document identifiers. Similarity retrieval procedure consists of the single query to the LSH index.

Evaluation of the index shows that with article 10 as query there are two candidates - article 10 (which is the query itself) and article 2952 (which is the nearest neighbor according to MinHash). The coincidence between MinHash result and LSH candidates shows that the indexing procedure preserves the highly similar documents while significantly decreasing the comparisons.

LSH increases scalability greatly compared to exhaustive search making the query time almost sublinear relative to the number of documents. Even though LSH does not guarantee finding every similar document, it provides a good compromise between accuracy and efficiency for large document collections.

5.4 TF-IDF and Cosine Similarity

Apart from the set-based approaches, TF-IDF with Cosine Similarity uses term frequencies and their importance. After preprocessing, the abstracts are represented as TF-IDF vectors using TF-IDF Vectorizer from scikit-learn, producing the high-dimensional sparse document-term matrix. All pairwise similarities are calculated between all documents.

For 16,787 documents, the full similarity matrix includes 140,893,291 pairs. The matrix is quite sparse: only 16,793,354 pairs have nonzero cosine similarity, which is 11.92% of all possible pairs. The maximum similarity value is 1.0, and several documents (for example, 5099, 6171, 6376) have identical or very similar abstracts, confirming the correctness of both the TF-IDF representation and cosine similarity calculation.

TF-IDF with Cosine Similarity provides a more informative representation than the set-based methods by considering both lexical overlap and term importance, and it is widely used as a baseline in document retrieval systems.

6. Experiments and Results

The methods were tested on the 16,787 New York Times article abstracts. The tests include verification of implementation correctness and algorithm performance, as well as qualitative analysis of selected representatives to compare the quality of the retrieved neighbors.

6.1 Exact Jaccard Similarity

The first experiment verifies the exact Jaccard implementation. The similarity between selected document pairs is calculated from their token sets after preprocessing.

For example, articles 10 and 10532 have $J = 0.142857$, indicating that about 14% of the vocabulary overlaps after preprocessing. Such low overlap is expected, as articles on related topics can use different named entities and specific terms. This shows that the Jaccard implementation works correctly and is suitable as a baseline for other approximations.

6.2 MinHash Approximation

The second experiment is the validation of the MinHash implementation. The MinHash signatures were produced for all 16,787 documents using 128 permutations. For the query document (article 10) the closest document is found as article 2952 with the estimated similarity of 0.203125.

The manual check of the retrieved documents showed a high overlap in token sets, thus confirming the validity of the approximation. The manual check indicator in the notebook confirms that the estimated similarity agrees with the textual similarity of the retrieved documents.

From a performance standpoint, generating MinHash signatures for the entire corpus took about 13 minutes and 2 seconds. This preprocessing cost is paid only once, and then similarity queries become much faster than exhaustive Jaccard comparison.

Overall, the results show that MinHash provides a good approximation to the exact Jaccard similarity while significantly reducing the cost of subsequent queries.

6.3 Locality Sensitive Hashing

The third experiment evaluates the MinHash-based LSH index. Instead of comparing the query with all documents, the index gives a set of candidates.

Thus, for the query document article 10, the candidates are article 10 and article 2952. The agreement between the LSH result and the MinHash nearest neighbor indicates the consistency of the approach. LSH significantly reduces the number of comparisons and provides nearly sublinear query performance as the dataset grows; hence, it is useful for large collections.

6.4 TF-IDF and Cosine Similarity

For the last experiment, document similarity search based on TF-IDF vectorization and Cosine Similarity was performed.

After preprocessing, all article abstracts were converted into TF-IDF vectors, yielding a sparse document-term matrix. Cosine similarities were then calculated for the whole dataset. With 16,787 documents in the dataset, there are 140,893,291 possible document pairs. However, the vast majority of these pairs have zero similarity because their corresponding documents lack informative vocabulary after preprocessing. The experiments showed that 16,793,354 document pairs had a non-zero cosine similarity. This means the proportion of such pairs was approximately 11.92%.

It shows the sparsity of the document collection and demonstrates the benefits of using sparse matrices for similarity computation in large textual corpora. The maximum value of the cosine similarity in the experiments was equal to 1.0.

It appeared that among these document pairs, several had identical or nearly identical processed abstracts. They included articles 5099, 6171, and 6376. It is expected that sometimes the same news is repeated in the New York Times. Overall, TF-IDF, along with Cosine Similarity, provided adequate rankings of document similarity and used sparse vector representations for efficient calculations.

7. Scalability Analysis

One of the aims of this project was not only the identification of similar documents but also the assessment of the performance of different similarity-calculation algorithms as the size of the document collection increased. That is why scalability is an important criterion for comparison of the methods.

Exact Jaccard Similarity allows calculating the most accurate measure of lexical similarity as the complete sets of tokens are compared. However, it incurs significant computational overhead, as every document must be compared with every other, so the number of comparisons grows quadratically with dataset size. As a result, the exhaustive Jaccard Similarity becomes unfeasible when the dataset consists of tens or hundreds of thousands of documents.

MinHash helps overcome the problem of quadratic growth in computation by replacing full token sets with smaller signatures. While generating signatures still requires initial preprocessing, it significantly reduces the computational cost of similarity estimation. Experiments with the New York Times dataset showed that it takes approximately 13 minutes to generate MinHash signatures for all 16,787 documents, and that subsequent similarity calculations are faster than exact Jaccard.

Locality Sensitive Hashing further improves scalability by skipping unnecessary comparisons. Instead of analyzing all the documents, LSH identifies only a small number of documents that are similar to the query document. Thus, it considerably reduces query time and enables similarity searches on larger datasets.

The TF-IDF approach has its own computational characteristics. First of all, generating the TF-IDF matrix involves one-time processing of each document, and cosine similarities are calculated efficiently using sparse matrix operations from scikit-learn. Although the entire similarity matrix contains more than 140 million document pairs, only 11.92% of them have non-zero similarity. It significantly decreases both memory usage and computational costs.

Table 1 summarizes the main characteristics of the implemented approaches.

Method	Representation	Accuracy	Scalability	Main Limitation
Jaccard Similarity	Sets of unique words	Exact	Low	Quadratic number of comparisons
MinHash	Compact signatures	Approximate	High	Initial signature generation
MinHash + LSH	Indexed MinHash signatures	Approximate	Very High	Possible omission of some similar documents
TF-IDF + Cosine Similarity	Sparse weighted vectors	High	High	Large sparse matrix for very large corpora

Overall, the experiments show that approximate similarity search algorithms can achieve considerable efficiency while maintaining high search quality. Among the implemented methods, the combination of MinHash and LSH offers the best scalability for large document collections, while TF-IDF with Cosine Similarity provides a richer semantic representation of textual content.

8. Discussion

The experiments conducted in this project highlight important features of document similarity search.

First, preprocessing proved crucial for calculating document similarity, as it allowed the removal of stop words, punctuation, and grammatical variations, and enabled estimation of similarity based on their semantic content.

Second, the experiments highlight the balance between exactness and efficiency of similarity search algorithms. The Jaccard Similarity provides an interpretable measure of lexical similarity between documents, but it is computationally expensive because it requires comparing every pair of documents. It motivates the usage of the approximate algorithms.

MinHash provided a good approximation of Jaccard Similarity while requiring only compact signatures per document. The retrieved nearest neighbors correlated well with manually inspected articles, demonstrating that the approximation retains the essential lexical properties of the documents.

The LSH index reduced the computational costs by limiting the number of documents for analysis to a small set of candidate documents. The fact that the same nearest neighbor was retrieved with MinHash similarity search and with the LSH index supports the validity of the implementation.

The TF-IDF representation provided the richest similarity rankings, taking into account not only shared vocabulary but also the weight of particular words. The experiments confirmed that most document pairs had a cosine similarity of 0, reflecting the high sparsity of the news corpus, and also helped identify identical or nearly identical articles with a cosine similarity of 1.

Each implemented approach has its own advantages, depending on the task at hand. Exact Jaccard Similarity is good for small datasets and validation of the methods. MinHash and LSH are especially useful for approximate similarity search in large-scale datasets, while TF-IDF with Cosine Similarity remains a good baseline for document retrieval tasks.

9. Conclusion

The purpose of the project is to investigate various techniques used for measuring document similarity across a large corpus of New York Times article abstracts. In particular, four algorithms are used – Exact Jaccard Similarity, MinHash, Locality Sensitive Hashing, and TF-IDF together with Cosine Similarity.

In all cases, each algorithm can find similar documents. They exhibit different computational properties. While Exact Jaccard Similarity provides the clearest understanding of lexical similarity between the two texts, it becomes increasingly computationally complex as the number of documents in the collection increases. On the other hand, MinHash significantly reduces computation time by replacing the actual set of tokens with shorter signatures, with little loss in approximation quality. Finally, Locality Sensitive Hashing helps reduce computation even further, since only potentially similar documents are compared for similarity.

Using TF-IDF with Cosine Similarity yielded meaningful similarity scores that account for both lexical and term-importance similarities between the two documents. The obtained similarity matrix is sparse, indicating that most document pairs in the news corpus are dissimilar; meanwhile, duplicates and

similar documents are properly identified. Overall, the goals of the project were successfully achieved, as several techniques for conducting effective large-scale similarity searches have been developed.

There is potential for future work since it would be interesting to implement context-aware language models such as Sentence-BERT and compare the results with TF-IDF + Cosine Similarity. It would be interesting to implement a FAISS vector database for similarity search in very large document collections.

I (Anastasia Khromova) declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work, and including any code produced using generative AI systems. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me/us or any other person for assessment on this or any other course of study.